

PRD - SYSTEM ROLE Arquiteto Mestre de Infraestrutura Cognitiva SmartHotel (ZEHLA)

SYSTEM OBJECTIVE *Arquitetar, compilar e fornecer o Documento de Requisitos do Produto (PRD) estrutural e exaustivo para a reconstrução limpa (Greenfield Reboot) do ecossistema SmartHotel/ZEHLA, mapeando todas as fases de engenharia, matrizes de LLMs, políticas de contenção e o arcabouço de variáveis de ambiente (.env) para garantir o isolamento contra os colapsos do sistema legado.*

SYSTEM CONTEXT O histórico da aplicação sofreu degradação por "Saturação de Contexto" (Agent Drift), vazamentos trans-locatários por ausência de Scope Locking e choques de compatibilidade interna (ex: CrewAI falhando na busca por OpenAI e Pydantic V2 colidindo com desempacotamentos YAML). Para reiniciar o projeto de forma saudável e determinística no Google Antigravity, a máquina precisa de um documento mestre absoluto (Single Source of Truth) que guie a IA sem exigir a retenção de contexto volátil do chat. Este PRD projeta o futuro do sistema e prevê todas as rotas operacionais e infraestruturais.

SKILL REGISTRY SKILL NAME: generate_master_prd_architecture Description: Compila o projeto sistêmico integral, das fundações de rede até a governança cognitiva, estabelecendo diretrizes infalíveis para os agentes da IDE. Inputs: historical_errors, smarthotel_documentation_library Outputs: PRD_SMARTHOTEL_V2.md Dependencies: GSD Framework, Zero-Trust Architecture Allowed Agents: ZEHLA Master Architect

SYSTEM MODULES

- Ingress & Webhook Layer (FastAPI + Evolution API).
- Event Kernel (Redis Streams + Inngest).
- Data & Memory Layer (Supabase PostgreSQL + pgvector + Semantic Cache).
- Cognitive Orchestration (LangGraph + CrewAI Dual Loop).
- Agent Tools & Skills (GWS CLI, Visual Analysis, Guest Retrieval).

AGENT STRUCTURE Origin: ZEHLA Master Architect Target: Operador Humano (Engenheiro Matriz) Fluxo Subsequente: O agente Google Antigravity lerá este PRD para executar o projeto de forma autônoma e segura.

DATABASE STRUCTURE Estrutura Multi-Tenant isolada rigorosamente por Row-Level Security (RLS) no PostgreSQL e pre-filtros de metadados (property_id) em buscas vetoriais no pgvector dimensionado em 384 dimensões.

SERVICE TASKS

- Especificar a arquitetura base para o Reboot.
- Mapear a topologia exata das chaves criptográficas para o .env.

3. Definir o sequenciamento tático de Fases (1 a 7) para o Agente da IDE seguir cegamente.
4. Estruturar a estratégia de "Skills" isoladas.

SKILL ORCHESTRATION O Operador Humano deve copiar o conteúdo gerado dentro de "IMPLEMENTATION NOTES", salvá-lo como um arquivo Markdown (`PRD_SMARTHOTEL_V2.md`) e subi-lo nas fontes do projeto. Quando o Agente IDE for reiniciado, o primeiro comando será a leitura deste manifesto, amordaçando seu comportamento aos padrões técnicos aqui definidos.

EVENT FLOWS Event: human_requests_reboot_blueprint -> Action: Architect generates PRD -> Event: human_saves_prd -> Action: IDE_Agent_ingests_prd -> System State: Safe Greenfield Execution Initiated.

IMPLEMENTATION NOTES

Copie o conteúdo abaixo na íntegra, salve-o como `PRD_SMARTHOTEL_V2.md` e adicione às fontes do projeto. Ele será a bússola inquebrável da máquina.



PRD MESTRE - SMARTHOTEL / ZEHLA (REBOOT V2)

Status: Aprovado para Implementação Greenfield. **Arquitetura:** SaaS Multi-Tenant, Serverless, Agent-First, Event-Driven.

1. VISÃO GERAL DO PROJETO

O ZEHLA (Zonal Entity for Hospitality Logic & Automation) é a camada de inteligência conversacional do ecossistema SmartHotel. Seu objetivo é gerenciar autonomamente a comunicação com hóspedes via WhatsApp, coordenar reservas, pagamentos e operações hoteleiras. O sistema abandona a arquitetura reativa antiga em prol de uma Malha de Controle Cíclica (Dual Loop) orquestrada por Agentes Autônomos (CrewAI) baseada em Grafos de Estado (LangGraph).

2. MATRIZ DE MODELOS (LLMs) E CHAVES DE AMBIENTE (.env)

A arquitetura prevê a utilização de múltiplos motores de inferência para balancear Custo (FinOps), Velocidade e Precisão. Todas as ferramentas e módulos devem buscar estritamente estas variáveis no `.env`.

O arquivo **.env** base deve ser criado com a seguinte topologia:

```
# [ 1. COGNITIVE ENGINE (MOTORES PRINCIPAIS) ]  
# Utilizado pelo orquestrador CrewAI (Manager Agent) e análise visual  
GEMINI_API_KEY="sua_chave_gemini_aqui"  
  
# Utilizado para varreduras de alto volume e ETL de históricos do WhatsApp (Rápido e barato)  
GROQ_API_KEY="sua_chave_groq_aqui"  
  
# Alternativa de back-up (se necessário para formatação JSON barata de RAG)  
OPENAI_API_KEY="sua_chave_openai_aqui"  
  
# [ 2. DATA & MEMORY LAYER (INFRAESTRUTURA) ]  
# Supabase: Única fonte da verdade (PostgreSQL relacional + pgvector para memória)  
SUPABASE_URL="sua_url_supabase_aqui"  
SUPABASE_SERVICE_ROLE_KEY="sua_chave_service_role_aqui"  
  
# [ 3. EVENT KERNEL & CACHE ]  
# Redis: Barramento de mensagens para webhooks e Cache Semântico  
REDIS_URL="sua_url_redis_aqui"  
  
# [ 4. INGRESS LAYER ]  
# Evolution API: Gateway de conexão com o WhatsApp  
EVOLUTION_API_URL="sua_url_evolution_aqui"  
EVOLUTION_API_KEY="sua_chave_evolution_aqui"
```

Estratégia de Roteamento de Modelos:

- **Gemini 1.5 Pro:** Cérebro do **Manager Agent** para raciocínio complexo e quebra de tarefas.
- **Gemini 1.5 Flash:** Utilizado para o "Fast Loop" (respostas rápidas e triviais).
- **Llama 3.1 8B/70B (via Groq):** Empregado estritamente no pipeline de **ingestion.py** para varrer anos de históricos de chat de donos de hotéis em segundos, barateando o *setup* inicial.
- **GPT-4o-mini:** Utilizado em processos de formatação tipada Pydantic caso o Gemini falhe em consistência de JSON.

3. AS 7 FASES DE ENGENHARIA (O ROTEIRO DE VOO)

O agente da IDE Google Antigravity DEVE executar o projeto seguindo estritamente esta ordem, sendo proibido de avançar para a fase seguinte sem a validação (**VERIFICATION.md**) da fase atual.

FASE 1: Fundação, Governança e Vacinas Arquiteturais

- **Objetivo:** Imunizar a base de código contra erros do passado.
- **Ações:**
 1. Criar a árvore de pastas (`api/`, `brain/`, `rag/`, `events/`, `tools/`, `database/`, `.agent/rules/`).
 2. Instanciar o `requirements.txt` utilizando o pacote `uv` para compilação estrita.
 3. Criar as **Rules**:
 - `isolation_policy.md`: Proibir varredura fora do diretório raiz.
 - `crewai_pydantic_policy.md`: Forçar a injeção manual de chaves (`role=config['role']`) e proibir o uso de empacotamento `config=` para evitar erros do Pydantic V2. Forçar injeção de `llm=self.manager_llm` em TODOS os agentes para evitar o "OpenAI Fallback" fatal.

FASE 2: Data & Memory Layer (A Única Fonte da Verdade)

- **Objetivo:** Consolidar o Supabase como núcleo, abandonando bancos locais desconexos (ex: ChromaDB).
- **Ações:**
 1. Criar `database/connection.py` com clientes assíncronos. Garantir configuração *Singleton* para evitar exaustão de conexões no webhook.
 2. Projetar as tabelas base (`guests`, `reservations`, `knowledge_base`).
 3. Implementar PostgreSQL Row-Level Security (RLS) baseada em `property_id` injetada via variáveis de sessão, garantindo isolamento Multi-Tenant.
 4. Configurar a tabela de memória vetorial explicitamente para `vector(384)`, correspondente ao modelo de embeddings `all-MiniLM-L6-v2`.

FASE 3: Ingress e Event Kernel (Assincronicidade e Debounce)

- **Objetivo:** Suportar rajadas de mensagens do WhatsApp sem derrubar o sistema.
- **Ações:**
 1. Configurar o endpoint FastAPI (`api/webhook_router.py`) retornando HTTP 200 em milissegundos para a Evolution API.
 2. Publicar o payload cru no Redis Streams.
 3. Implementar o worker do Inngest com a função de **Debounce** de 4 a 60 segundos baseada na chave `remote_jid + property_id`. Isso agrupa mensagens fragmentadas do hóspede ("Oi" -> "Quero cancelar" -> "o quarto 402") em um único evento analítico, salvando milhares de tokens.

FASE 4: Orquestração Cognitiva (LangGraph + CrewAI)

- **Objetivo:** O cérebro do ZEHLA.

- **Ações:**
 1. Projetar a máquina de estados no LangGraph (`brain/engine.py`). Se a intenção for simples (ex: "Qual a senha do wifi?"), acionar o **Fast Path** (consulta determinística). Se for complexa, direcionar ao **Slow Path** (CrewAI).
 2. Corrigir o erro crítico C1: Nenhuma chamada a `asyncio.create_task()` deve ocorrer dentro de nós síncronos do LangGraph. Toda função que acessa o banco deve ser nativamente `async`.
 3. Construir a topologia Hierárquica do CrewAI (`brain/crew.py`): O **Manager Agent** (Gemini 1.5 Pro) delega tarefas ao **Guest Concierge**, **Reservation Guardian** e **Billing Specialist**.

FASE 5: Agent Tools e Sensory Extensibility

- **Objetivo:** Conceder "mãos e olhos" aos agentes autônomos.
- **Ações:**
 1. Desenvolver a `GuestRetrievalTool` usando Pydantic `BaseModel`. É OBRIGATÓRIO o uso da variável `property_id` na classe para forçar o limite locatário na extração de dados.
 2. Integrar o `VisualAnalysisTool` usando o Google Gemini nativamente multimodal, capaz de fatiar imagens (Dynamic Tiling) de recibos e quartos para análise.
 3. Integrar a **GWS CLI** (Google Workspace CLI) executada via túnel subprocess assíncrono para interações seguras e sem middlewares com Planilhas, Documentos e Calendários do administrador do hotel.

FASE 6: Matriz de Segurança Zero-Trust e Privacidade (PII)

- **Objetivo:** Blindar o hotel contra ataques de prompt e vazamento de dados.
- **Ações:**
 1. Instalar grades de proteção (Guardrails) na camada do LangGraph para interceptar injeções de prompt no payload do WhatsApp.
 2. Para processamento de comprovantes PIX ou documentos de hóspedes (multimodalidade), aplicar o conceito de Processamento Efêmero com Retenção Zero (Burn-After-Use). Nenhuma mídia ou PII (Personal Identifiable Information) deve ser gravada no disco, mitigando passivos regulatórios (LGPD).

FASE 7: Telemetria e Validação Local (End-to-End Test)

- **Objetivo:** Provar a sanidade funcional do código.
- **Ações:**
 1. Criar scripts de simulação local (`test_agent.py`) que acionem o `zehla_kickoff()` injetando o `session_id` simulado, forçando o log do processo de pensamento (Chain-of-Thought) do agente diretamente no terminal do host.

2. Auditoria visual dos artefatos produzidos.
-

4. ESTRUTURA DE RETENÇÃO (GSD E SKILLS)

Para evitar que o modelo do Antigravity colapse tentando processar todos os manuais simultaneamente, implementaremos a **Divulgação Progressiva**.

- As lógicas pesadas de "como acessar a API da Evolution" ou "como fazer chamadas GWS CLI" não devem residir no prompt geral do sistema.
 - Devem ser fragmentadas e encapsuladas na pasta `.agent/skills/`.
 - Cada habilidade terá um manifesto `SKILL.md` com um cabeçalho YAML contendo as "trigger phrases". O agente as lerá apenas no exato milissegundo em que perceber a necessidade daquela habilidade específica, poupando a janela de tokens de alucinações e degradação progressiva.
-

Fim do Documento. Aguardando ingestão pelo Agente de Infraestrutura.